

Calculadora Web con Python y Flask

Construcción paso a paso de una aplicación web utilizando Python y Flask, desde la creación del proyecto hasta su publicación en Internet.

Capítulo 1

Entorno y Renderizado de una página web

Objetivo: Establecer la arquitectura inicial del proyecto;
Configurar un entorno de desarrollo aislado;
Comprender el ciclo de vida de una solicitud web.

1.1 Estructura inicial

1.2 Entorno virtual

1.3 Dependencias

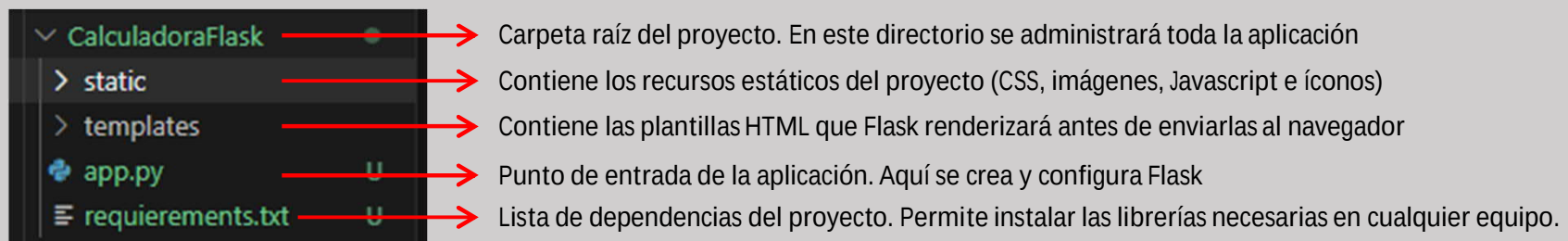
1.4 Aplicación Flask

1.5 Gestión del proyecto con .gitignore y requirements.txt

1.1 Creación de la estructura inicial del Proyecto

Objetivo: Crear la estructura inicial del proyecto, siguiendo la organización recomendada por Flask.

1.1 Estructura inicial



¿Por qué organizar correctamente un proyecto?

- Facilita el mantenimiento del código
- Permite que otras personas comprendan el proyecto rápidamente
- Sigue las buenas prácticas de programación
- Simplifica el despliegue de la aplicación

1.2 Entorno virtual

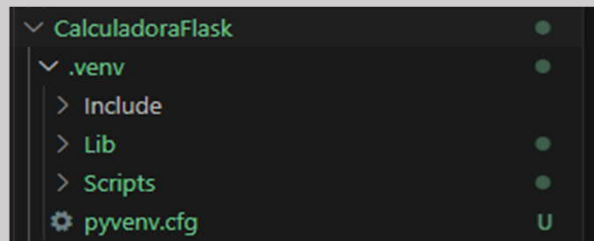
Objetivo: Crear un entorno virtual para aislar las dependencias del proyecto

1.2 Entorno virtual

¿Qué hace este comando?

Crea un entorno virtual independiente que contendrá las librerías utilizadas por este proyecto. De esta manera, se evita que diferentes proyectos compartan dependencias o generen conflictos entre versiones.

```
Windows PowerShell
PS D:\0_LearningPython\0_LearningPython_LP\CalculadoraFlask> python -m venv .venv
```



Resultado

Al ejecutar este comando se crea la carpeta `.venv`, que contiene el entorno virtual y todos los archivos necesarios para administrar las dependencias del proyecto.

¿Qué hace este comando?

Activa el entorno virtual y lo notas porque al inicio de la línea de comando aparece `(.venv)`

```
PS D:\0_LearningPython\0_LearningPython_LP\CalculadoraFlask> .\.venv\Scripts\activate
(.venv) PS D:\0_LearningPython\0_LearningPython_LP\CalculadoraFlask> pip list
Package Version
-----
pip      24.0
```

1.4 Aplicación Flask

Objetivo: Crear la primera aplicación web utilizando Flask y comprender como Python responde a las solicitudes de un navegador.

1.4 Nuestra primera aplicación Flask

```
app.py U x
0_LearningPython_LP > CalculadoraFlask > app.py > ...
1  from flask import Flask
2
3  app = Flask(__name__)
4
5  @app.route("/")
6
7  def inicio():
8      return "<h1>¡Hola Mundo!</h1>"
9
10 app.run()
```

¿Qué hace esta aplicación?

app.py

- Este programa crea una aplicación web con Flask, inicia un servidor local y queda esperando solicitudes provenientes de un navegador. Cuando un usuario accede a la página principal, Python ejecuta la función inicio() y devuelve la respuesta correspondiente.

¿Qué sucede al ejecutar este comando?

python app.py

- Se inicia la aplicación Flask y Python comienza a ejecutar un servidor web local. Mientras la terminal permanezca abierta, el servidor estará disponible para recibir solicitudes del navegador.
- El mensaje GET / indica que el navegador solicitó la página principal de la aplicación.
- El mensaje GET /favicon.ico corresponde al intento automático del navegador por obtener el ícono del sitio. Como todavía no existe, Flask responde con un código 404, lo cual es completamente normal durante el desarrollo.

Resultado

- Al escribir la dirección <http://127.0.0.1:5000> El navegador envía una solicitud a Flask.
- Flask ejecuta la función asociada a la ruta principal (/) y devuelve al navegador el contenido generado por Python.

```
Windows PowerShell
(.venv) PS D:\0_LearningPython\0_LearningPython_LP\CalculadoraFlask>
python app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production
deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [02/Jul/2026 12:42:02] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [02/Jul/2026 12:42:02] "GET /favicon.ico HTTP/1.1" 404
```



1.5 Gestión del proyecto con .gitignore y requirements.txt

Objetivo: Comprender cómo **.gitignore** y **requirements.txt** permiten compartir un proyecto Python correctamente mediante Git.

¿Qué aprenderemos?

- Comprender qué archivos deben formar parte del repositorio Git.
- Aprender el propósito de **.gitignore**.
- Comprender cómo **requirements.txt** permite reconstruir el proyecto en cualquier computadora.

Archivos que se suben a git

Se sube a Git	No se sube a Git
app.py	.venv/
templates/	pycache/
static/	*.pyc
requirements.txt	.env
README.md	archivos temporales

Si el entorno virtual (**.venv**) se sube a Git, al clonar el proyecto en otra computadora pueden aparecer errores como:

Fatal error in launcher
Unable to create process...



¿Qué va dentro del archivo?
.gitignore

```
C: > 0_LearningPython > CalculadoraFlask > .gitignore
1 # Entorno virtual
2 .venv/
3
4 # Caché de Python
5 __pycache__/
6 *.py[cod]
7
8 # Variables de entorno
9 .env
10
11 # Archivos de VS Code (opcionales)
12 .vscode/
13
14 # Archivos del sistema
15 .DS_Store
16 Thumbs.db
```

```
<> PowerShell
git rm -r --cached .venv
```

Elimina el entorno virtual del repositorio sin borrarlo del disco. (Si por error se agregó)

¿Qué va dentro del archivo?
requirements.txt

```
C: > 0_LearningPython > CalculadoraFlask
You, 20 minutes ago | 1 author (Yo
1 blinker==1.9.0
2 click==8.4.2
3 colorama==0.4.6
4 Flask==3.1.3
5 itsdangerous==2.2.0
6 Jinja2==3.1.6
7 MarkupSafe==3.0.3
8 Werkzeug==3.1.8
```

```
<> PowerShell
pip freeze > requirements.txt
```

Actualiza la lista de dependencias del proyecto.

requirements.txt almacena la lista de todas las librerías y sus versiones utilizadas por el proyecto. Gracias a este archivo, cualquier desarrollador puede recrear el mismo entorno de desarrollo ejecutando:

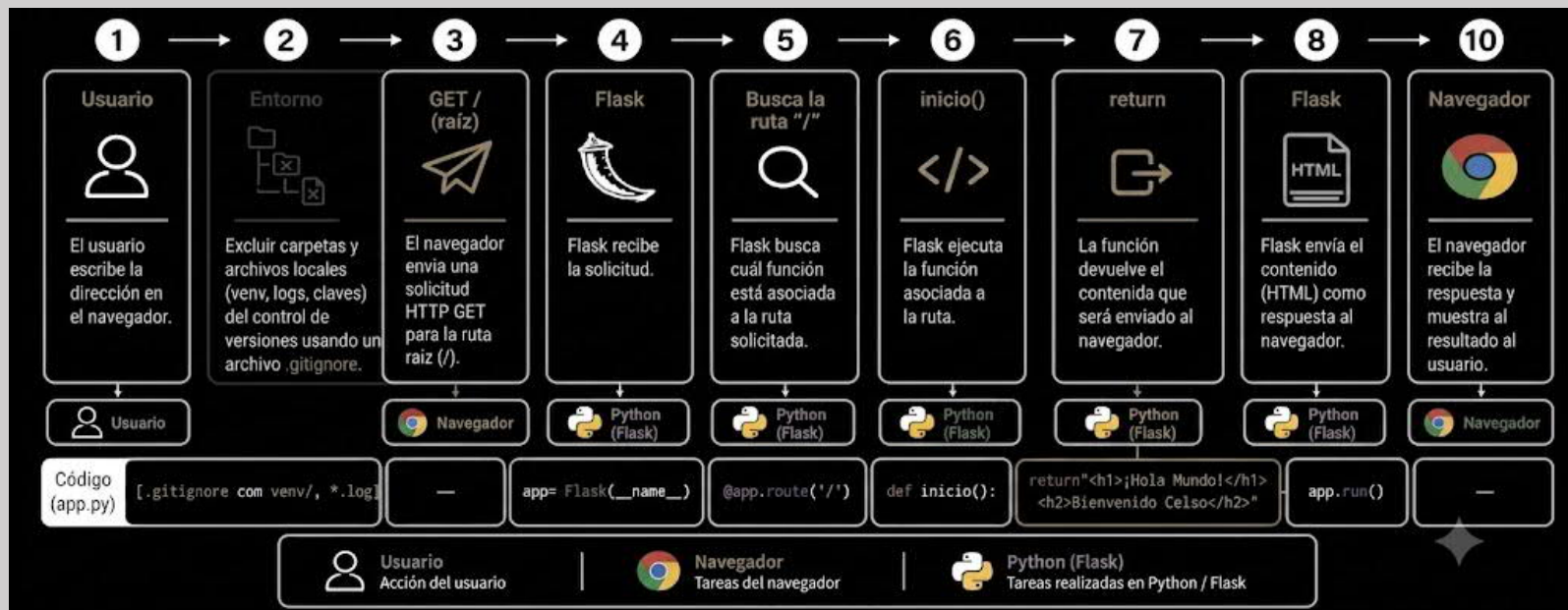
```
<> PowerShell
pip install -r requirements.txt
```

Resumen del capítulo 1

Objetivo: Crear la primera aplicación web utilizando Flask y comprender como Python responde a las solicitudes de un navegador.

Capítulo 1 – Resumen Después de este capítulo ya sabemos:

- Crear la estructura recomendada para un proyecto Flask.
- Aislar dependencias mediante un entorno virtual.
- Instalar librerías y registrar sus versiones.
- Ejecutar una aplicación Flask local.
- Comprender el flujo básico entre navegador, Flask y Python.
- Excluir carpetas que no se subirán al sistema usando **.gitignore**



Capítulo 2

Templates y páginas HTML

Objetivo: Aprender a separar el contenido visual de la lógica Python,
Usando archivos HTML dentro de la carpeta templates.

2.1 ¿Por qué no devolver HTML directamente desde Python?

2.2 Crear index.html dentro de templates

2.3 Usar `render_template()`

2.4 Mostrar la calculadora en una página web

2.1 ¿Por qué no devolver HTML desde Python?

Objetivo: Comprender por qué en Flask es recomendable separar la lógica (Python) de la presentación (HTML).

Python también está escribiendo la página

```
@app.route("/")

#Lengaje HTML escrito directamente en Python
def inicio():
    return """
    <html>
      <body>
        <h1>¡Hola Mundo!</h1>
        <p> Bienvenido Celso</p>
      </body>
    </html>
    """
```

En proyectos pequeños puedes devolver HTML desde Python, pero en proyectos profesionales se separan ambas responsabilidades.

Arquitectura correcta

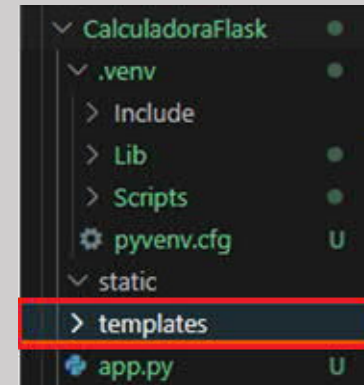
```
Python
-----
Procesa datos
Aplica reglas
Toma decisiones

↓

HTML
-----
Muestra información
Define el diseño
Recibe formularios
```

Ventajas de separar el código

- Código más limpio.
- HTML fácil de modificar.
- Permite al diseñador trabajar en paralelo.
- Facilita el mantenimiento del proyecto.

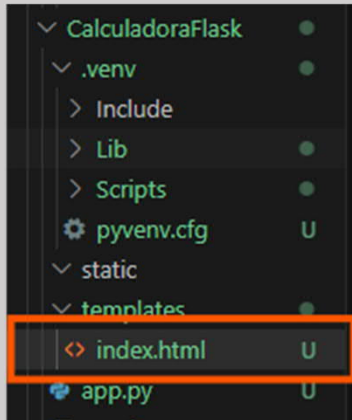


```
Proyecto pequeño
↓
Python + HTML

Proyecto profesional
↓
Python + HTML separados
```

2.2 Creación de la primera plantilla HTML

Objetivo: Crear la primera plantilla HTML que Flask enviará al navegador.



```
<? index.html U •
0_LearningPython_LP > CalculadoraFlask > templates > <? index.html > ...
1
2 <!--Primer archivo creado en HTML
3     Se encuentra separado dentro de la carpeta templates -->
4
5 <!DOCTYPE html>
6 <html lang="es">
7     <head>
8         <meta charset="UTF-8">
9         <title>Calculadora Flask - Primera página Flask</title>
10    </head>
11
12    <body>
13
14        <h1> Mi primera plantilla </h1>
15        <h2> Calculadora Flask </h2>
16
17    </body>
18
19 </html>
```

¿Qué es una plantilla HTML?

- Una plantilla HTML es un archivo que contiene la estructura visual de una página web y que Flask enviará al navegador cuando sea necesario.

Ventajas

- El contenido visual queda separado del código Python.
- La página puede modificarse sin alterar la lógica de la aplicación.
- Permite reutilizar la misma estructura en diferentes páginas.

2.3 Renderizar la primera plantilla HTML

Objetivo: Aprender cómo Flask utiliza `render_template()` para enviar una plantilla HTML al navegador para construir la página que verá el usuario.

Antes

```
app.py U X index.html U
0_LearningPython_LP > CalculadoraFlask > app.py > ...
1 from flask import Flask
2
3 app = Flask(__name__)
4
5 @app.route("/")
6
7 #Lengaje HTML escrito directamente en Python
8
9 def inicio():
10     return """
11     <html>
12     <body>
13     <h1>¡Hola Mundo!</h1>
14     <p> Bienvenido Celso</p>
15     </body>
16     </html>
17     """
18
19 app.run()
```

Archivo de Python (app.py)

Es importante que el código se encuentre fuera de la carpeta **templates**

Ahora

```
app.py U index.html U
0_LearningPython_LP > CalculadoraFlask > app.py > ...
1 from flask import Flask, render_template
2
3 app = Flask(__name__)
4
5 @app.route("/")
6
7 #Flask enviará el contenido de index.html
8
9 def inicio():
10     return render_template("index.html")
11
12 app.run()
13
```

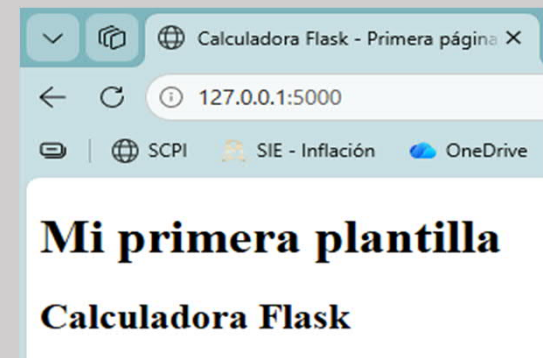
¿Qué hace `render_template()`?

Busca el archivo HTML dentro de la carpeta **templates** y lo envía al navegador.

index.html

```
app.py U index.html U X
0_LearningPython_LP > CalculadoraFlask > templates > index.html > ...
1
2 <!--Primer archivo creado en HTML
3 Se encuentra separado dentro de la carpeta templates -->
4
5 <!DOCTYPE html>
6 <html lang="es">
7     <head>
8         <meta charset="UTF-8">
9         <title>Calculadora Flask - Primera página Flask</title>
10     </head>
11     <body>
12
13         <h1> Mi primera plantilla </h1>
14         <h2> Calculadora Flask </h2>
15
16     </body>
17 </html>
```

Resultado



2.4 Construyendo la primera interfaz de usuario

Objetivo: Construir la primera interfaz web de la aplicación, permitiendo que el navegador capture información para enviarla posteriormente a Python

¿Qué construiremos?

- La aplicación dejará mostrar únicamente información y comenzará a presentar una interfaz donde el usuario podrá capturar datos para interactuar con Python

Tabla de componentes

Elemento	Función
 Título	Identifica la aplicación.
 Valor 1	Captura el primer número.
 Valor 2	Captura el segundo número.
 Botón	Envía la información a Python para su procesamiento.
 Resultado	Muestra la respuesta generada por la aplicación.

Interfaz

Cálculadora básica

Primer reto de Python llevado a la web con Flask

Valor 1

Valor 2

Resultado:

Código en index.html

```
index.html U • app.py U •
0_LearningPython_LP > CalculadoraFlask > templates > index.html > html > body
1
2 <!--Primer archivo creado en HTML
3 Se encuentra separado dentro de la carpeta templates -->
4
5 <!DOCTYPE html>
6 <html lang="es">
7   <head>
8     <meta charset="UTF-8">
9     <title>Calculadora Flask - Primera página Flask</title>
10  </head>
11  <body>
12    <h1> Cálculadora básica </h1>
13    <h2> Primer reto de Python llevado a la web con Flask </h2>
14    <label >Valor 1</label>
15    <input type = "number" placeholder="Ingresa el primer número">
16    <br><br>
17    <label >Valor 2</label>
18    <input type = "number" placeholder="Ingresa el segundo número">
19    <br><br>
20    <button>Calcular</button>
21    <p>Resultado:</p>
22  </body>
23 </html>
```

¿Por qué necesitamos una interfaz?

Porque el usuario necesita introducir información para que Python pueda procesarla.

Estado del proyecto al momento

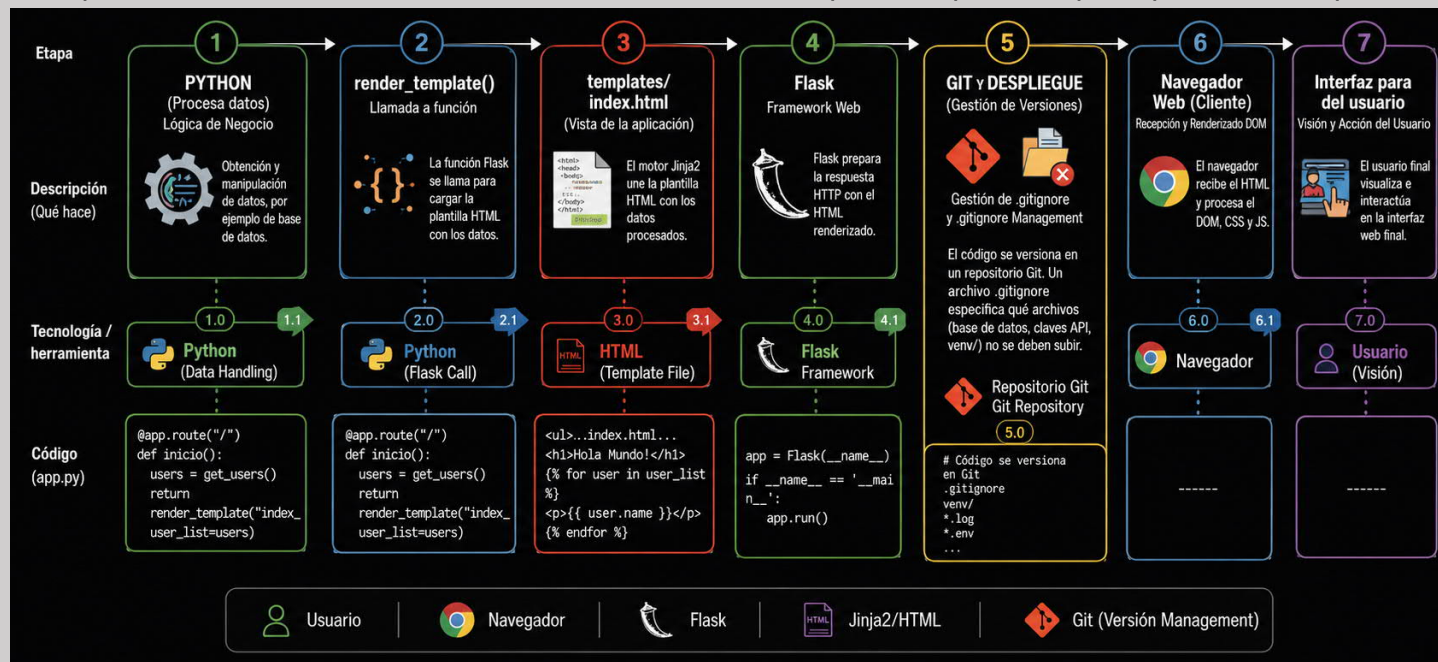
✓ Proyecto creado	✓ templates
✓ Entorno virtual	✓ Renderización HTML
✓ Flask instalado	✓ Primera interfaz web
✓ Primera aplicación	■ Comunicación con Python

Resumen del capítulo 2

Objetivo: Aprender a separar la presentación (HTML) de la lógica (Python), utilizando las plantillas HTML administradas por Flask

Capítulo 1 – Resumen Después de este capítulo ya sabemos:

- Comprender porque Python y HTML deben mantenerse separados
- Utilizar la carpeta templates para almacenar las páginas HTML
- Crear nuestra primera plantilla index.html
- Utilizar `render_template()` para enviar una plantilla al navegador
- Construir la primera interfaz gráfica de la aplicación
- Preparar la aplicación para que el usuario pueda interactuar con Python



Capítulo 3

Comunicación entre el navegador y Python

Objetivo: Aprender como la información viaja desde el navegador hacia Python para ser procesada y cómo el resultado regresa nuevamente a la página web.

3.1 Enviar información desde la interfaz

3.2 Procesar la información en Python

3.3 Mostrar el resultado en la página web

3.1 Enviar información desde la interfaz

Objetivo: Modificar la interfaz HTML para que el navegador pueda enviar los datos capturados hacia Flask.

¿Qué construiremos?

- La interfaz dejará de ser únicamente visual y comenzará a enviar la información introducida por el usuario hacia la aplicación Flask.

Elementos que agregaremos

Elemento	Función
<code><form></code>	Agrupar todos los controles que enviarán información.
<code>method="POST"</code>	Indica que los datos serán enviados al servidor.
<code>action="/"</code>	Especifica la ruta de Flask que recibirá la información.
<code>name</code>	Permite identificar cada dato enviado desde el formulario.

Antes

La interfaz solo mostraba información

Ahora

La interfaz también puede enviar información a Flask

Interfaz

Calculadora básica 📊

Primer reto de Python llevado a la web con Flask

Valor 1

Valor 2

Resultado:

Los datos ingresados se envían a Python (Flask) para procesar el cálculo.

Campos de entrada
El usuario ingresa datos que serán enviados a Python.

Botón de acción
Envía los datos a Python para realizar el cálculo.

Resultado
Python procesa los datos y devuelve el resultado.

Nota: Los elementos resaltados son los que interactúan con Python (Flask).

Ahora estos controles formarán parte de un formulario capaz de enviar información a Flask.

Elementos a cambiar

```
3 <form method="post" action="/">
4
5 <input type="number"
6     id="valor1" name="valor1">
7
8 <input type="number"
9     id="valor2" name="valor2">
10
11 <button type="submit"
12     id="calcula" name="calcula">Calculate</button>
13
14 </form>
```

Identificamos los elementos y les damos nombre para poder trabajar con estos en el código de Python

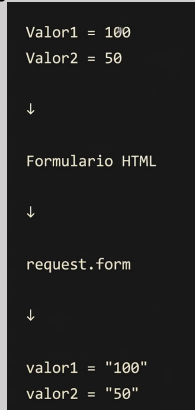
3.2 Procesar la información en Python

Objetivo: Recibir en Python la información enviada desde el formulario HTML y convertirla en variables que puedan utilizarse dentro del programa.

¿Qué construiremos?

- Flask recibirá los datos enviados desde el navegador mediante una petición POST.
- Los valores capturados en la interfaz se convertirán en variables de Python listas para ser procesadas.

Flujo de la información



¿Qué ocurrió?

Los datos dejaron de estar únicamente en la página web y ahora existen como variables dentro del programa Python.

Código de Python

```
templates > app2.py > ...
...
1 from flask import Flask, request
2 from flask import Flask, render_template
3
4 app = Flask(__name__)
5
6 @app.route("/", methods=["GET", "POST"])
7
8 def inicio():
9     if request.method == "POST":
10         valor1 = request.form["valor1"]
11         valor2 = request.form["valor2"]
12         # Process the submitted values
13         return f"Values submitted: {valor1}, {valor2}"
14     return render_template("index2.html")
15
16 app.run()
```


request.form permite recuperar los valores enviados desde el formulario utilizando el nombre definido en el atributo name del HTML.

Estado de la aplicación

El depurador muestra el estado de la aplicación. En la sección 'VARIABLES', bajo 'Locals', se muestra el diccionario de valores recibidos: `(return) ImmutableMultiDict.__getitem__ = '50'`, `> (return) cached_property.__get__ = ImmutableMultiDict(valor1 = '100', valor2 = '50')`. En la sección 'WATCH', se muestran los valores de las variables: `valor1 = '100'` y `valor2 = '50'`.

El depurador confirma que Flask recibió correctamente la información enviada desde el navegador.

3.3 Mostrar el resultado en la página web

Objetivo: Enviar el resultado calculado por Python nuevamente al navegador para que el usuario pueda visualizarlo en la misma página web.

¿Qué construiremos?

- Python realizará el cálculo utilizando las variables recibidas.
- Flask enviará el resultado junto con la plantilla HTML.
- El navegador actualizará la página mostrando el resultado al usuario.

Flujo del resultado

```
valor1 = 100
valor2 = 50

↓

resultado = 150

↓

render_template()

↓

resultado = 150

↓

Página HTML
```

Python calcula el resultado y Flask genera la respuesta
HTML

```
app2.py > ...
1  from flask import Flask, request
2  from flask import Flask, render_template
3  from os import system
4
5  system("cls")
6
7  app = Flask(__name__)
8
9  @app.route("/", methods=["GET", "POST"])
10
11  def inicio():
12      if request.method == "POST":
13          valor1 = request.form["valor1"]
14          valor2 = request.form["valor2"]
15          resultado = float(valor1) + float(valor2)
16          # Process the submitted values
17          return f"Values submitted: {valor1}, {valor2}. \
18          <br> <br> Sum Result: {resultado}"
19          return render_template("index2.html")
20
21  app.run()
```

Resultado mostrado en el navegador

Values submitted: 100, 50.
Sum Result: 150.0

El navegador recibe una nueva página HTML ya construida por Flask, donde el resultado calculado por Python ya forma parte del contenido mostrado al usuario.

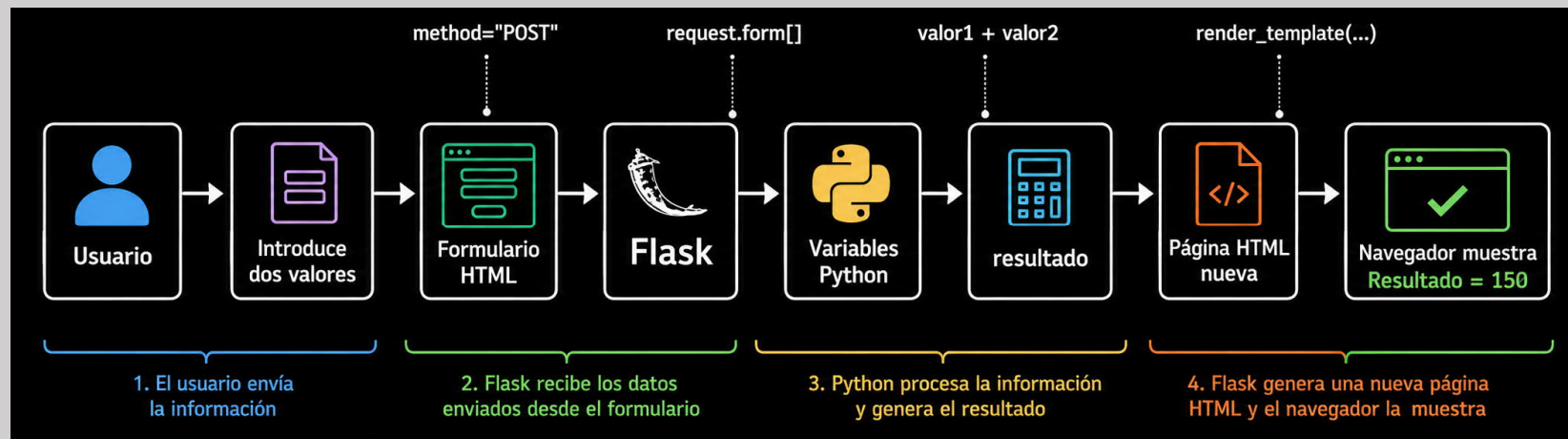
Aquí ya estamos trabajando con la información recibida por el usuario dentro del HTML `render_template()` también puede enviar variables hacia HTML. Flask incorpora esas variables al renderizar la página para que el navegador pueda mostrarlas al usuario.

Resumen del capítulo 3

Objetivo: Comprender el ciclo completo mediante el cual una aplicación Flask recibe información del navegador, la procesa en Python y devuelve el resultado nuevamente al usuario.

Capítulo 3 – Resumen Después de este capítulo ya sabemos:

- Construir un formulario HTML capaz de enviar información mediante POST.
- Recuperar los datos enviados utilizando `request.form`.
- Procesar la información dentro del programa Python.
- Generar un resultado utilizando variables de Python.
- Enviar variables desde Flask hacia HTML mediante `render_template()`.
- Mostrar el resultado actualizado en la misma página web.



Capítulo 4

Objetivo: -----

4.1 Preparar el proyecto para GitHub

4.2 Publicar el repositorio en GitHub

4.3 -----

4.4 -----